

Advanced Software Engineering

Course Syllabus & Roadmap — Fall 2026 (115-1)

Graduate Seminar (進階軟體工程)

Instructor: Prof. Nien-Lin Hsueh (薛念林 教授)

Department of Information Engineering and Computer Science

Feng Chia University

Course Overview & Philosophy

- **Bridging Theory, Modern Toolchains, and AI:**
 - Software Engineering is evolving rapidly from manual implementation to **AI-assisted & spec-driven engineering**.
 - This course balances **classical engineering rigor** (processes, architecture, quality assurance) with **modern paradigms** (generative AI, automated pipelines, empirical analysis).
- **Key Course Attributes:**
 - **Target Audience:** Graduate students and motivated senior undergraduates.
 - **Language of Instruction:** English-friendly / bilingual materials.
 - **Interactive Pedagogy:** Concept Check Questions (CCQ) with real-time polling, architectural critiques, and collaborative discussions.

Course Learning Objectives (CLOs)

- **1. Master Core SE Disciplines:**

- Deeply understand software lifecycles, requirement engineering, object-oriented modeling, and architectural styles.

- **2. Quality-First & Empirical Mindset:**

- Internalize testing methodologies (black-box, white-box), CI/CD automation, and empirical experimentation.

- **3. Responsible & Rigorous AI Integration:**

- Learn how to effectively steer AI coding agents, validate AI-generated code, and mitigate hallucinations and technical debt.

- **4. Scholarly & Professional Communication:**

- Deconstruct peer-reviewed SE literature, run conference-style panels, and deliver production-ready software engineering post-mortems.

Curriculum Roadmap: Five Core Modules

- **Module 1: Foundations (Weeks 1)**
 - From the 1968 Software Crisis to Software Engineering; From Software to Qualified Software.
- **Module 2: Processes & Requirements (Weeks 2–4)**
 - Agile Methodologies, Scrum, DevOps, Spec-Driven AI workflows.
 - Requirements Engineering (Functional/Non-Functional Specs, AI Spec)
- **Module 3: Software Design & Modeling (Weeks 5–9)**
 - Design Principles & Patterns, UML Modeling, Software Architectures.
 - Mid-term exam.
- **Module 4: Quality Assurance & Testing (Weeks 10–13)**
 - Testing Fundamentals & Practices: Black-box testing, white-box testing, coverage criteria.
 - Quality Assurance Model & Process Improvement Model
- **Module 5: Empirical Software Engineering & Capstone Execution (Weeks 14–16)**
 - Controlled experiments, final capstone demos and panel debates.

Week 3 (09/28) & Week 7 (10/26): National Holiday

Grading Scheme

- **25% — Midterm Exam**
 - Part I Evaluation (25%)
- **25% — Final Exam**
 - Part II Evaluation (25%)
- **20% — In-Class Engagement & QA**
 - Active questioning during panel and pitches
- **30% — Final Capstone Project**
 - Execution of Track 1, Track 2, or Track 3

Final Capstone Project: Three Flexible Tracks

- **Team-Based Project:** Performed by teams of **~4 members**.
- **Presentation Duration:** Time period = **25 min × number of members** (e.g., ~100 min for 4 members).
- **Track 1: Academic Paper Panel Discussion**
 - Conducted in a **panel discussion style** to present peer-reviewed papers (*ICSE, FSE, ASE, TSE, TOSEM*).
 - The team acts as panelists (~4 persons) deconstructing papers, synthesizing trade-offs, and running Q&A.
- **Track 2: Software System Prototype & SE Post-Mortem**
 - Live demo + in-depth SE post-mortem focusing on a **thin vertical slice**.
 - Architectural separation, AI-assisted workflow logs, CI/CD pipeline, and automated tests.
- **Track 3: Empirical Software Engineering Experiment**
 - Scientific presentation + replication package (dataset & benchmark scripts).
 - Formulate research questions (RQ_1, RQ_2), evaluate tools/models, and analyze threats to validity.

Track 1: Academic Paper Panel — Illustrative Themes

- **Theme Example A: "Security & Trust in Generative AI Code Assistants"**
 - **Panelist 1 (ICSE):** Prevalence and taxonomy of CWE security flaws in Copilot-generated code.
 - **Panelist 2 (FSE):** Automated hallucination detection and differential testing for LLM unit tests.
 - **Panelist 3 (ASE):** Developer over-reliance and cognitive bias during AI-assisted code reviews.
 - **Panelist 4 (TOSEM):** Code licensing contamination and privacy leakages in training corpuses.
 - **Moderator Synthesis:** Guiding the debate on enterprise AI governance and developer accountability.
- **Theme Example B: "Microservices Resilience & Modern DevOps Automation"**
 - **Panelists 1 & 2 (ICSE/FSE):** Automated fault injection, Chaos Engineering, and flaky test mitigation.
 - **Panelists 3 & 4 (ASE/TSE):** Machine learning for CI/CD test selection and architectural debt prediction.
 - **Moderator Synthesis:** Balancing rapid continuous deployment velocity against architectural stability.

Track 2: System Prototype & Post-Mortem — Examples

- **System Example A: "Intelligent Clinical Triage & Patient Routing Service"**

- **Thin Vertical Slice:** Intake form → AI structured triage categorization → priority assignment engine → physician notification webhook (secondary EHR / billing mocked).
- **Engineering Rigor:** Clean Architecture (Domain / Use Cases / Adapters), schema validation with strict Pydantic/Zod models, Dockerized local testing environment.
- **Post-Mortem Focus:** AI hallucination guardrails, defensive exception handling, and 90%+ test coverage.

- **System Example B: "Automated Git PR Code Review & Compliance Gatekeeper"**

- **Thin Vertical Slice:** GitHub Webhook trigger → AST diff parser → LLM rule-checking engine (security & style guidelines) → automated PR comments & status check.
- **Engineering Rigor:** Event-driven architecture, rate-limiting & asynchronous job workers, mocked GitHub API.
- **Post-Mortem Focus:** Prompt engineering reproducibility, handling false positives, and latency trade-offs.

Track 3: Empirical SE Experiment — Examples

- **Study Example A: "Benchmarking Open-Source vs. Commercial Coding LLMs"**
 - **Objective:** Controlled benchmark evaluating code correctness and security across models.
 - **Research Questions:**
 - RQ_1 : Do open-source models (e.g., DeepSeek-Coder, Llama-Code) inject higher rates of CWE flaws than GPT-4o on HumanEval-Security?
 - RQ_2 : How does test-driven iterative prompting affect pass@k accuracy and token costs?
 - **Deliverables:** Curated benchmark datasets, automated evaluation scripts, and replication Docker repo.
- **Study Example B: "Empirical Evaluation of Automated Program Repair (APR) in CI"**
 - **Objective:** Measure real-world patch correctness vs. plausible test-overfitting across APR tools.
 - **Research Questions:**
 - RQ_1 : What percentage of LLM-generated bug fixes are semantically sound vs. merely overfitting?
 - RQ_2 : How does token context window depth impact multi-file bug repair success?
 - **Deliverables:** Defects4J benchmark replication suite, execution telemetry logs, and threat analysis.

Academic Integrity & Generative AI Policy

- **Generative AI is Welcomed, but Must Be Steered Responsibly:**
 - You are encouraged to use AI tools (e.g., Claude, ChatGPT, GitHub Copilot, Cursor).
 - **Rule of Transparency:** Document where and how AI was utilized (prompts, edits, test generation).
- **The "Accountability" Rule:**
 - **You are 100% responsible** for the correctness, security, and architectural integrity of your submissions. Blindly accepting hallucinated code or fabricated paper citations results in severe penalties.
- **Plagiarism & Authorship:**
 - Uncredited copying of text, diagrams, or code from external repositories or human authors is strictly prohibited.

References & Recommended Resources

- **Core References:**

- Ian Sommerville, *Software Engineering*, 10th Edition, Pearson.
- Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
- Selected top-tier research papers from *ACM/IEEE ICSE*, *IEEE TSE*, *ACM TOSEM*.

- **Digital Toolchains:**

- **Version Control & CI/CD:** GitHub, GitHub Actions, Docker.
- **Interactive Polling:** NickEduPocket CCQ System ([nickedupocket](#)).
- **Slide & Handout Toolchain:** Marp Markdown Slide Engine.

Getting Started & Action Items for Week 1

- **Immediate Next Steps:**

1. Review the Three Capstone Tracks (Academic Panel, System Prototype, Empirical Study).
2. Brainstorm topics and start reaching out to potential teammates (teams of ~4).
3. Ensure each team incorporates at least one international/exchange student.
4. Bring your laptop/mobile device to class for interactive CCQ polls.

- **Course Inquiries & Office Hours:**

- **Instructor:** Prof. Nien-Lin Hsueh (薛念林 教授)
- **Office:** Department of Information Engineering and Computer Science
- **Contact & Appointments:** Via course LMS or email

Questions & Discussion

Welcome to Advanced Software Engineering!

Let's build reliable, elegant, and impactful software systems together.